

# Algorithms and Data Structures

## Problem Sheet 4

Hermann Schichl

44. **Searching an unsorted array 1** This problem examines an algorithm for searching for a value  $x$  in an unsorted array  $A$  consisting of  $n$  elements called `random_search`. Consider the following randomized strategy: pick a random index  $i$  into  $A$ . If  $A[i] = x$ , then terminate; otherwise, continue the search by picking a new random index into  $A$ . Continue picking random indices into  $A$  until you find an index  $j$  such that  $A[j] = x$  or until every element of  $A$  has been checked. This strategy may examine a given element more than once, because it picks from the whole set of indices each time.

In the following we assume that all possible permutations of the input array are equally likely.

- (a) Implement a procedure `random_search` to realise the strategy above. Be sure that your algorithm terminates when all indices into  $A$  have been picked.
  - (b) Suppose that there is exactly one index  $i$  such that  $A[i] = x$ . What is the expected number of indices into  $A$  that must be picked before  $x$  is found and `random_search` terminates?
  - (c) Suppose now that there are  $k > 1$  indices  $i$  such that  $A[i] = x$ . What is the expected number of indices into  $A$  that must be picked before  $x$  is found and `random_search` terminates? Your answer should be a function of  $n$  and  $k$ .
  - (d) Suppose that there are no indices  $i$  such that  $A[i] = x$ . What is the expected number of indices into  $A$  that must be picked before all elements of  $A$  have been checked and `random_search` terminates?
45. **Searching an unsorted array 2** This problem examine the search algorithm `deterministic_search`. This algorithm searches  $A$  for  $x$  in order, considering  $A[1]$ ,  $A[2]$ ,  $\dots$   $A[n]$ , until either it finds  $A[i] = x$  or it reaches the end of the array. In the following we again assume that all possible permutations of the input array are equally likely.
- (a) Implement a procedure `deterministic_search` to realise the strategy above.
  - (b) Suppose that there is exactly one index  $i$  such that  $A[i] = x$ . What is the average-case running time of `deterministic_search`? What is its worst-case running time?

- (c) Suppose that there are  $k > 1$  indices  $i$  such that  $A[i] = x$ . What is the average-case running time of `deterministic_search`? What is the worst-case running time of `deterministic_search`? Your answer should be a function of  $n$  and  $k$ .
- (d) Suppose that there are no indices  $i$  such that  $A[i] = x$ . What is the average-case running time of `deterministic_search`? What is its worst-case running time?

46. **Searching an unsorted array 3** This problem examines the search algorithm `scramble_search`. It first randomly permutes the input array and then runs the algorithm `deterministic_search` on the resulting permuted array. In the following we also assume that all possible permutations of the input array are equally likely.

- (a) Implement a procedure `scramble_search` to realise the strategy above. Use your implementation of `deterministic_search` from Exercise 45.
- (b) Suppose that there is exactly one index  $i$  such that  $A[i] = x$ . What is the expected number of indices into  $A$  that must be picked before  $x$  is found and `scramble_search` terminates?
- (c) Suppose now that there are  $k > 1$  indices  $i$  such that  $A[i] = x$ . What is the expected number of indices into  $A$  that must be picked before  $x$  is found and `scramble_search` terminates? Your answer should be a function of  $n$  and  $k$ .
- (d) Suppose that there are no indices  $i$  such that  $A[i] = x$ . What is the expected number of indices into  $A$  that must be picked before all elements of  $A$  have been checked and `scramble_search` terminates?

Which of the three searching algorithms `random_search`, `deterministic_search`, or `scramble_search` would you use and why?

**Scheduling** In the Linux system all processes gets a base priority value between 0 and 139. The process scheduler computes a real priority value from the process's base priority value. Lower priority values are better than higher priority values, i.e. from two processes the one with higher priority is the one with lower real priority value. Assume that the process scheduler always picks the next process among those with lowest real priority value. Assume further that every process has in addition to its two priority values the following additional information of the specified type: `process_id` (integer), `process_name` (string), `user_id` (integer), `group_id` (integer), `registers` (array(integer) of size 20), `process_flags` (array(boolean) of size 20), `floating_point_registers` (array(double) of size 16).

- 47. Implement a heap data structure that can be used for the priority queue as described above.
- 48. Implement a priority queue data structure for the process scheduler using the implemented heap of Exercise 47 and handles.
- 49. Implement a simple scheduler that performs a loop, such that every iteration performs the following algorithm:

- (a) Pick and remove the highest priority process  $P$  from the priority queue.
- (b) Decrease the real priority value of every process still in the queue by 1.
- (c) With a 10% chance discard the process  $P$  (it has terminated), with a 90% chance re-insert the process  $P$  into the queue with a new real priority value of 10 times its base priority value plus 1.
- (d) With a 10% chance randomly create a new process with random base priority in the range  $[1, 138]$  and put it in the queue with real priority value equal to its base priority value.

Then populate the priority queue with a process called “kernel” with process\_id 0, user\_id 0, group\_id 0, base priority value 1, the rest random, a process called “idle” with process\_id 1, user\_id 0, group\_id 0, base priority value 139, the rest random, and three other completely random processes. Then perform 1000 iterations of your process scheduler and print the process that was chosen in every iteration.

- 50. When RANDOMIZED-QUICKSORT runs, how many calls are made to the random-number generator RANDOM in the worst case? How about in the best case? Give your answer in terms of  $\Theta$ -notation.
- 51. Professors Howard, Fine, and Howard have proposed a deceptively simple sorting algorithm, named stooge sort in their honor.

```

1: procedure STOOGE-SORT( $A, p, r$ )
2:   if  $A[p] > A[r]$  then
3:     exchange  $A[p]$  with  $A[r]$ 
4:   end if
5:   if  $p + 1 < r$  then
6:      $k = \lfloor (r - p + 1)/3 \rfloor$ 
7:   end if
8:   STOOGE-SORT( $A, p, r - k$ ) // first two-thirds
9:   STOOGE-SORT( $A, p + k, r$ ) // last two-thirds
10:  STOOGE-SORT( $A, p, r - k$ ) // first two-thirds again
11: end procedure

```

- (a) Argue that the call STOOGE-SORT( $A, 1, n$ ) correctly sorts the array  $A[1 : n]$ .
- (b) Give a recurrence for the worst-case running time of STOOGE-SORT and a tight asymptotic ( $\Theta$ -notation) bound on the worst-case running time.
- (c) Compare the worst-case running time of STOOGE-SORT with that of insertion sort, merge sort, heapsort, and quicksort. Do the professors deserve tenure?